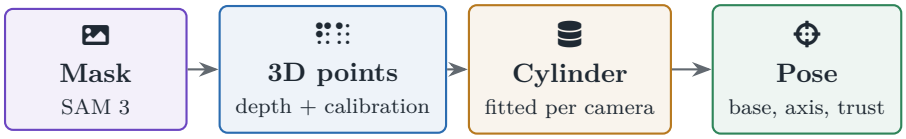
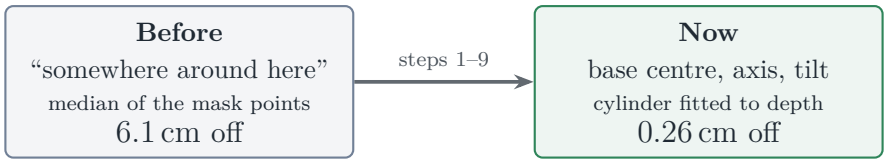
# Where Is the Mug?

Object poses from the depth cameras:
what we built in steps 1–9, why, and what it means for the supervisor




**Key idea**

The robot can now say *where the bottom of the mug is* to within a few millimetres, *which way it points* to about a degree, *where its handle is*, and *where on the rack's peg a hung mug must end up*, using only its cameras and its own joint sensors. It already stops the gripper opening on a mug that lies on its side. Behind a setting, it can also aim the arm at the mug and the rack.

# Contents

**1 Why the robot needs a pose** **2**
1.1 The rules we kept . . . 2

**2 The whole pipeline on one page** **3**
2.1 Category, not model . . . 4

**3 Step 1: measuring before improving** **4**
3.1 The answer key was wrong by two centimetres . . . 4
3.2 Which way is “up” for a mug model . . . 4

**4 Step 2: from depth to points and normals** **5**

**5 Step 3: fitting a cylinder** **6**
5.1 The trap, and how we avoided it . . . 7

**6 Removing the table, and a shadow** **7**

**7 Step 4: how much to trust a fit** **8**
7.1 Trusted and untrusted: an asymmetric rule . . . 8
7.2 More than one camera . . . 9

**8 Placing the base** **9**

**9 Step 5: the release check uses the axis** **9**

**10 Results of steps 1–5** **10**

**11 Step 6: which end is the base, which way is the handle** **11**
11.1 Two cues for the base end, which must agree . . . 12
11.2 Handle heading . . . 12

**12 Step 7: the rack from depth** **13**

**13 A second look: the code review** **13**

**14 Step 9: aiming the arm with the poses** **14**
14.1 What moves . . . 14
14.2 The peg seat: aim at what success is measured by . . . 15
14.3 The rack frame for the hang . . . 15

**15 Step 8: a clean test on fresh seeds** **16**
15.1 Against the targets set in the plan . . . 16
15.2 The four failures . . . 16
15.3 The release check on fresh seeds . . . 17
15.4 Which cameras a live run actually has . . . 17

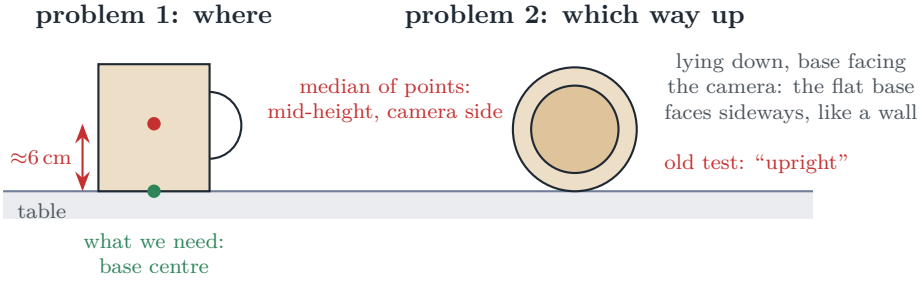
**16 What this means for supervising the task** **18**
16.1 Five things the supervisor gains . . . 18
16.2 How the supervisor should read a pose . . . 19
16.3 What the planner can and cannot infer . . . 19

**17 What is still open** **19**

**18 Commands** **20**

# 1 Why the robot needs a pose

Until now, "where is the mug" was answered with one number per axis: the **median** of the 3D points inside the mug's mask. That is a fine "roughly here", but it has two problems that matter for the supervisor.




1. **Position.** The median lands about half a mug height above the base and is pulled towards the camera. On seeds 0–4 it was **6.1 cm** from the true base centre for mugs on the table. A grasp or a placement aimed there is aimed at air.
2. **Orientation.** "Safe to open the gripper" (`release_ready`) checked that the mug's visible surface faces sideways, as an upright mug's wall does. A mug lying with its base towards the camera fools it, because a flat base also faces sideways. On held-out seed 6 this produced **25 wrong "safe to release"** answers.

**In plain words**

Knowing *where a cup is* is not enough to pick it up or put it down. You need to know where its bottom is and whether it is standing or lying. A pose is those two things together.

## 1.1 The rules we kept

**✔ Allowed at runtime**

RGB-D images and their calibration
SAM 3 masks
the robot's own measured joints,
hand poses and grippers

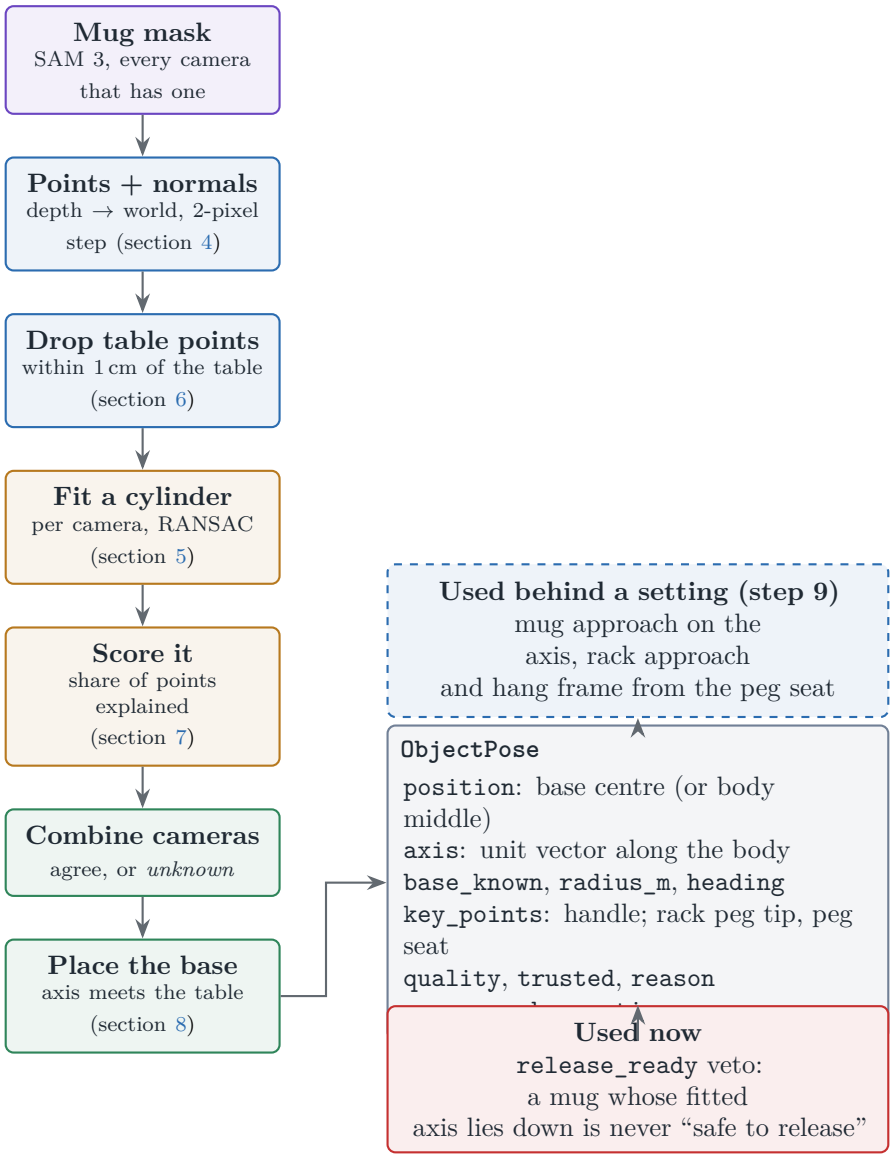
**✖ Offline only**

simulator object poses (labels)
the mug meshes
which mug model was loaded

**⚖ How we decide**

*unknown* beats a guess
precision before recall
tune on seeds 0–4, check on others

## 2 The whole pipeline on one page




### Where it lives in the code

- `harness/perception/object_pose.py`: everything in the left column, plus `ObjectSpec` and `ObjectPose`.
- `harness/perception/rgbd.py`: `RGBDCamera.surface_points_and_normals`.
- `harness/perception/grounding.py`: `RGBDPerceptionProvider._mug_pose` runs it once per observation; the result is `GroundingResult.mug_pose`.
- `development/replays/perception_evaluation.py`: pose metrics against the labels.

### 2.1 Category, not model

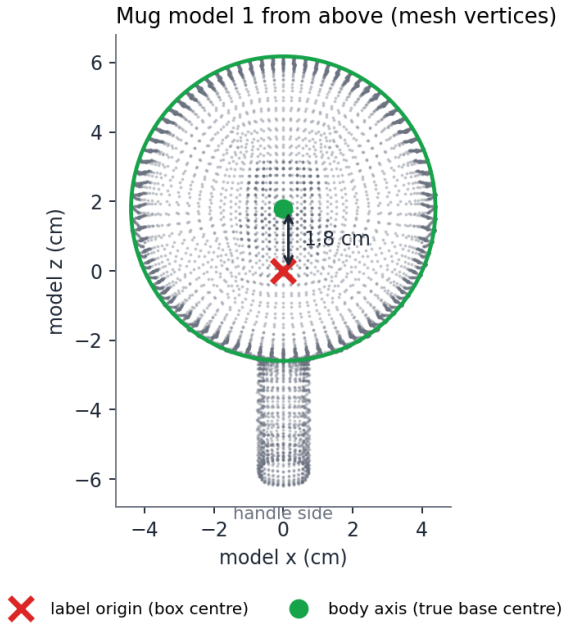
The fitter knows a mug only as an `ObjectSpec`: "a cylinder with radius 3–7 cm and height 6–14 cm that stands on the table". It never looks up which of RoboTwin's ten mug meshes was loaded. A different mug, or a bottle, needs a new spec line, not new code. This was one of the decisions in the plan: no mesh files at runtime.

## 3 Step 1: measuring before improving

Every change here was judged by numbers from the existing evaluator, which replays saved frames through the real runtime code and compares with the simulator's labels. We added pose metrics: axis error, base position error, upright/toppled mistakes, coverage, and **confident wrong poses** (trusted, but off by more than 20° or 3 cm). That last one is the pose version of a false "true".

### 3.1 The answer key was wrong by two centimetres

The first run gave a stubborn 1.9 cm horizontal error on *every* mug on the table, with almost no spread. A constant error is usually a definition problem, not a perception one.




**Careful**

The label's `mug.pose` origin is the centre of a box that *includes the handle*. The mug's body axis sits **1.6–2.6 cm** away from it, on the side away from the handle, depending on the model.

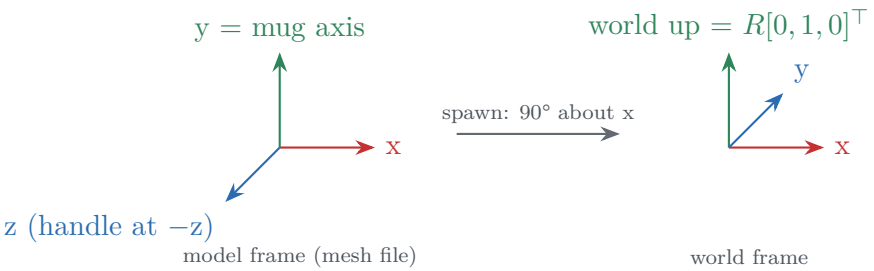
What we did (offline only):

- measured each model's offset from its mesh (`MUG_MODELS`);
- recognised the episode's model from its handle point, which is different for every model;
- moved the truth onto the body axis (`true_mug_base`);
- made new captures record `mug.model_id` directly.

Error after the fix: **0.26 cm**.

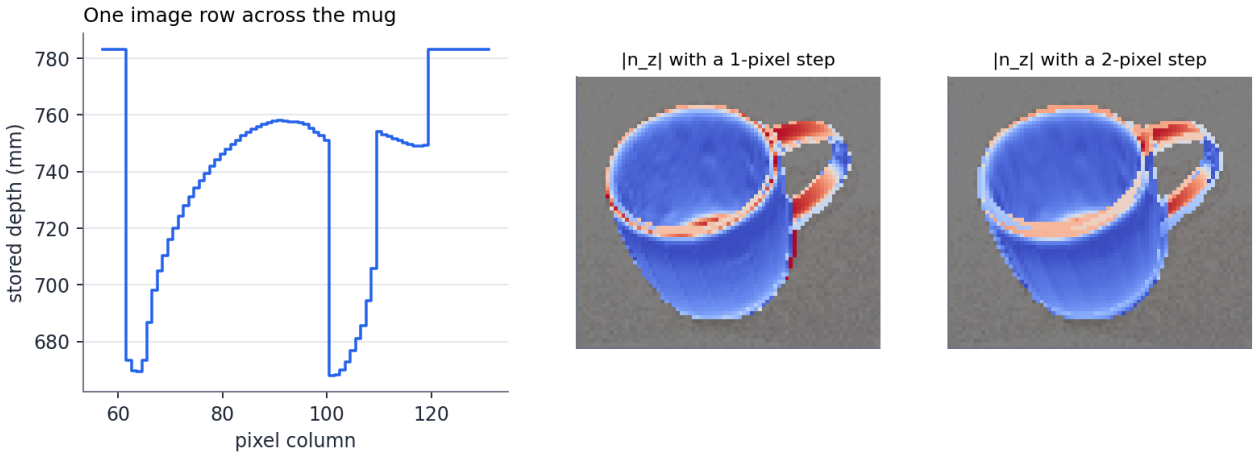
### 3.2 Which way is "up" for a mug model

The meshes stand along their **+y** axis. The task spawns them turned 90° about x, so +y points up in the world. The truth axis is therefore $R\,[0,1,0]^\top$, not the quaternion's z axis as the first draft of the plan said.



## 4 Step 2: from depth to points and normals

Each mask pixel with a depth becomes a world point (as before). We also need its **surface normal**: which way the surface faces. The normal at a pixel comes from the depth of its neighbours left/right and up/down.




- **Depth is stored in whole millimetres** (left: the staircase). Neighbours one pixel apart often differ by exactly 0 or 1 mm, which makes normals jumpy. Measuring across **two pixels** on each side averages that rounding out; the wall colour is a little more even on the right. We did not run a separate test of one step against the other.
- **A bug we hit:** pixels next to an edge are skipped when neighbouring depths jump by more than 1 cm. With the wider step, the two neighbours are 4 pixels apart, so a steep but smooth surface also "jumps". A toppled mug kept only **109 of its 1618 mask pixels**. The limit now grows with the step.

**Where it lives in the code**

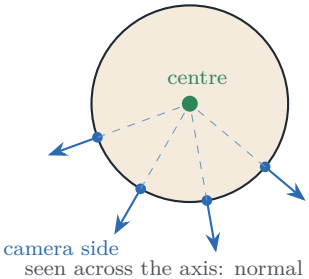
`RGBDCamera.surface_points_and_normals(mask, step_px=...)` returns world points and unit normals (turned to face the camera). The old `vertical_normal_components` now calls it, so the existing surface test is unchanged.

## 5 Step 3: fitting a cylinder

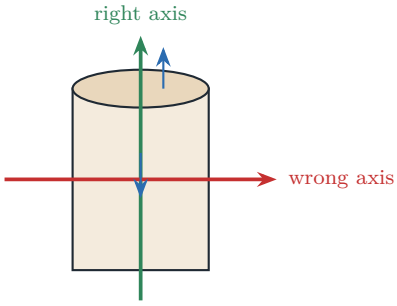
A mug's body is close to a cylinder. A cylinder has a simple property we can test point by point:

**Key idea**

On the curved wall, every surface normal is **perpendicular to the axis** and **points at the axis**. Two wall normals are enough to guess the axis (their cross product), and their lines meet at the centre.




seen across the axis: normal lines meet at the centre, even when only a narrow arc is visible

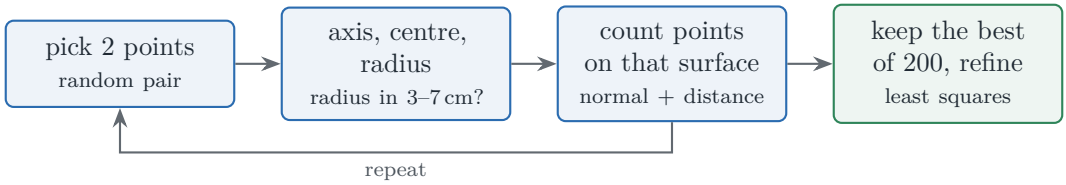



from above, the front wall (facing us) and the flat rim (facing up) are *both* perpendicular to a horizontal axis too

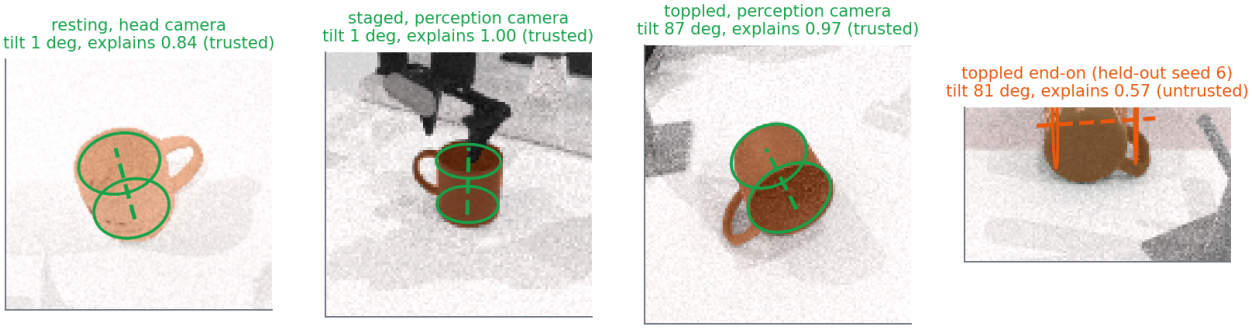
### 5.1 The trap, and how we avoided it

Our first version scored an axis guess only by "how many normals are perpendicular to it". From an elevated camera that let a **horizontal** axis explain an upright mug's front wall and rim (right drawing). In the first test on seeds 0–4, more than a dozen per-camera fits of upright mugs came out about 90° wrong.

The fix is to test a *whole cylinder*, not just a direction. Each random pair of points fixes the axis, the centre (where their normal lines meet) and the radius; a point only counts if its normal fits *and* it lies on that cylinder's surface (within 4 mm). Wrong axes stop adding up.



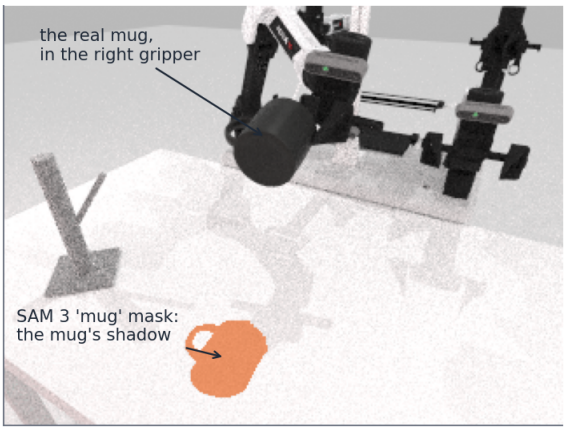

**End caps.** When a mug's flat base faces the camera squarely there is almost no wall to see. If most normals agree, the points are flat, and the patch is *round* with a mug-sized radius, it is an **end cap** and the axis is its normal. "Round" matters: a flat strip along a lying mug's side passed the first version of this test.




Fitted cylinders drawn back onto real frames (orange = SAM 3 mask). Right: the held-out end-on mug that fooled the old test. The fit reads it as lying down (81°), but it explains only 57% of its points, so it is not trusted (section 7).

## 6 Removing the table, and a shadow

Masks spill a little onto the table past the mug's edge. Points within 1 cm of the table top are dropped before fitting. This also fixed a detector mistake:



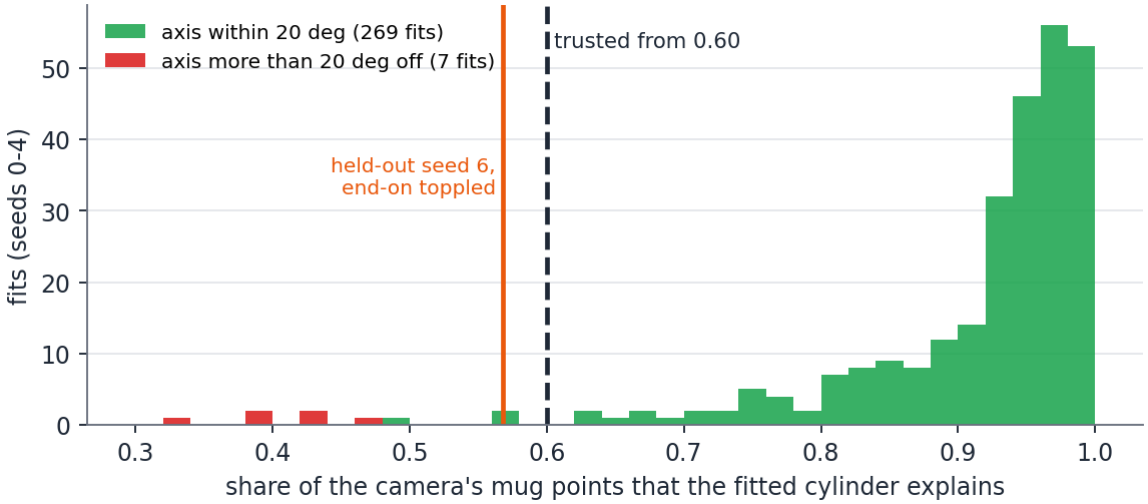

Here SAM 3 labelled the mug's **shadow** as the mug, while the real mug was in the right gripper. The shadow is flat and round, so it looked like an end cap standing upright.

Every shadow point lies on the table, so after the table points are removed nothing is left to fit, and the pose is *unknown*, which is the right answer.

The table height comes from the current frame's table mask, the same points the support check already uses.

# 7 Step 4: how much to trust a fit

A fit always returns *some* cylinder. We need a number that says whether to believe it. We use the share of the camera's mug points that the fitted cylinder **explains**: they lie on its wall, or on a flat end inside its radius (the base, or the floor inside the cup).

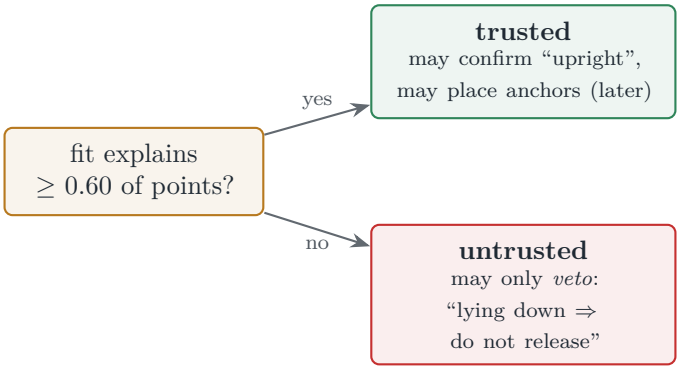



On seeds 0–4 the wrong fits (red, all mugs half hidden by a gripper) explain 47% of their points or less. Almost all right fits explain far more. The floor of **0.60** sits in that gap. This histogram uses every camera with a mug mask. In a real run the detector asks only some cameras each phase; for those fits, every wrong fit was below 0.44 and every right one above 0.70.

## 7.1 Trusted and untrusted: an asymmetric rule

The held-out end-on mug (orange line) was read correctly as lying down, but at 0.57 it fell just below the floor and was thrown away, so the old test let it through.

Lowering the floor until it passed would be tuning on the test set. Instead we applied the project's precision rule: **a reading that blocks an unsafe action needs less confidence than one that allows it**.

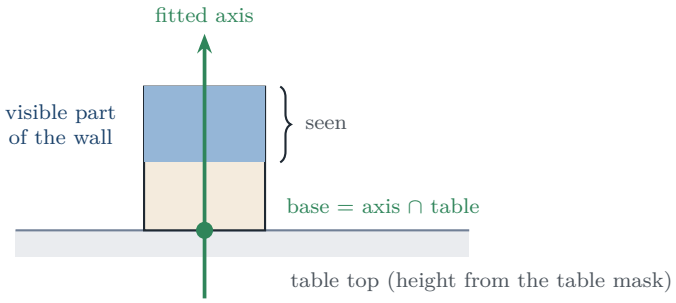



A wrong veto costs a little recall: the supervisor waits. A wrong "safe" drops the mug.

### 7.2 More than one camera

Each camera is fitted on its own. Trusted fits must agree (axes within 15°, axis lines within 3 cm). If they disagree the pose is *unknown* with reason `cameras_disagree`; we do not pick a favourite. The best-supported fit gives the pose.

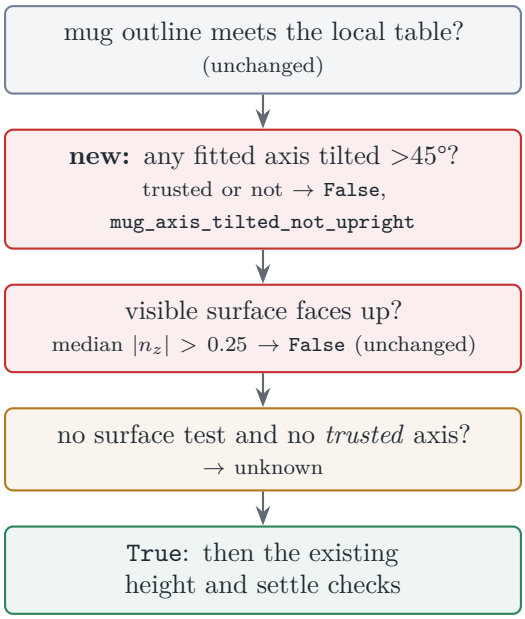
## 8 Placing the base




If the axis is within 45° of vertical (the same cut-off the labels use for "toppled") and the lowest visible part comes within 3 cm of the table, the mug stands on the table. Its base centre is then where the axis meets the table top, **even though the base itself is hidden**. That is what removed the 5.7 cm height error.

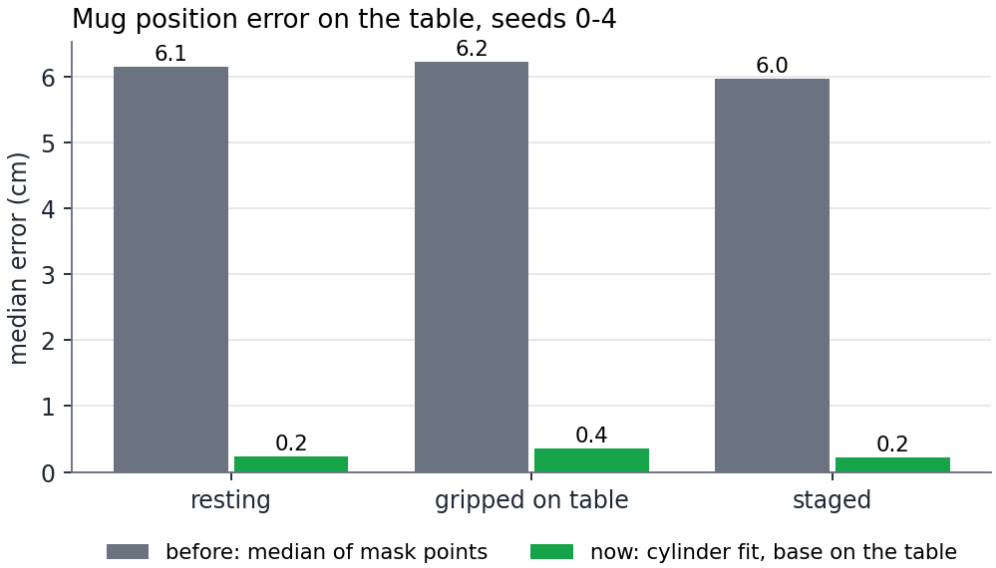
Otherwise (lying down, or lifted) `base_known` is false and `position` is the middle of the visible body. Which end of a lying mug is the base is step 6.

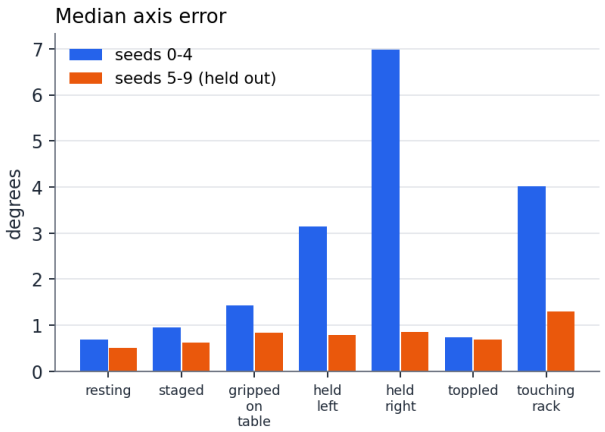
## 9 Step 5: the release check uses the axis

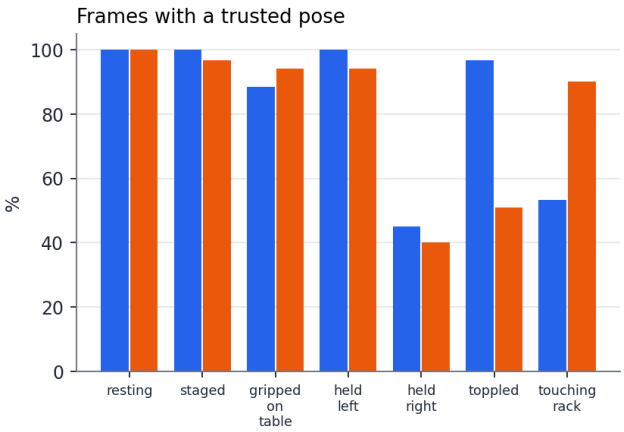


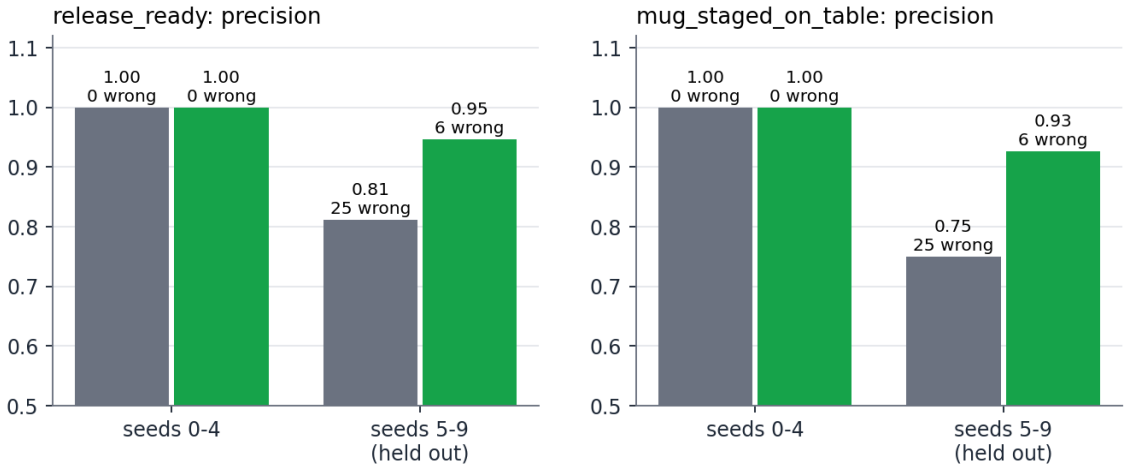

Either test can say "not upright". A trusted upright axis can also stand in for the surface test when the mask was too small for it (the old "sliver" case).

# 10 Results of steps 1–5



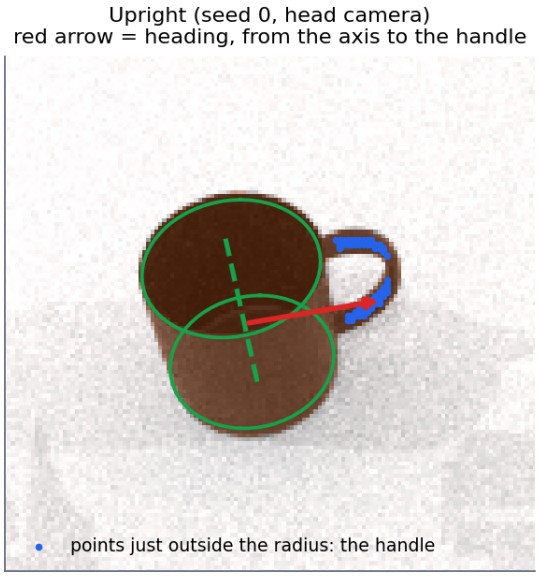



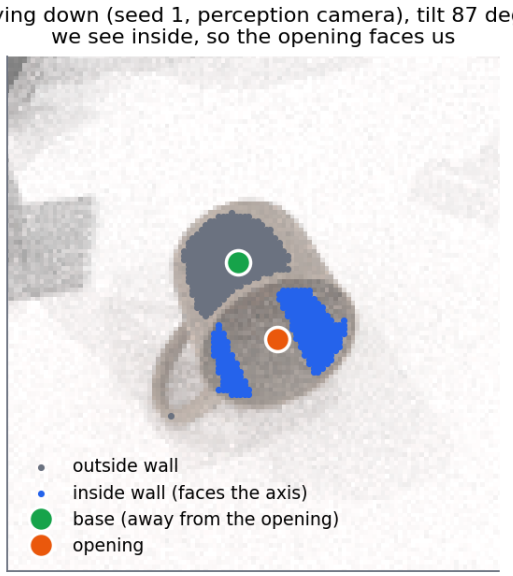
| Run | Posed | Axis error | Base error | Up/toppled mix-ups | Confident wrong |
|---|---|---|---|---|---|
| seeds 0–4, with perception camera | 159/216 | 1.1° | 2.6 mm | 0 | 0 |
| seeds 0–4, without it | 98/216 | 1.5° | 2.5 mm | 0 | 0 |
| seeds 5–9, with perception camera | 188/267 | 0.7° | 2.8 mm | 0 | 0 |
| seeds 5–9, without it | 157/267 | 0.7° | 2.7 mm | 0 | 0 |

- **No predicate lost anything**: every precision and recall on seeds 0–4 is identical to before.
- **Held-out "safe to release" false trues: 25 → 6**; precision 0.81 → 0.95.
- "Posed" counts trusted poses only. Most frames without one are mugs off the table or hidden by a gripper; the pose then says *unknown* with a reason.

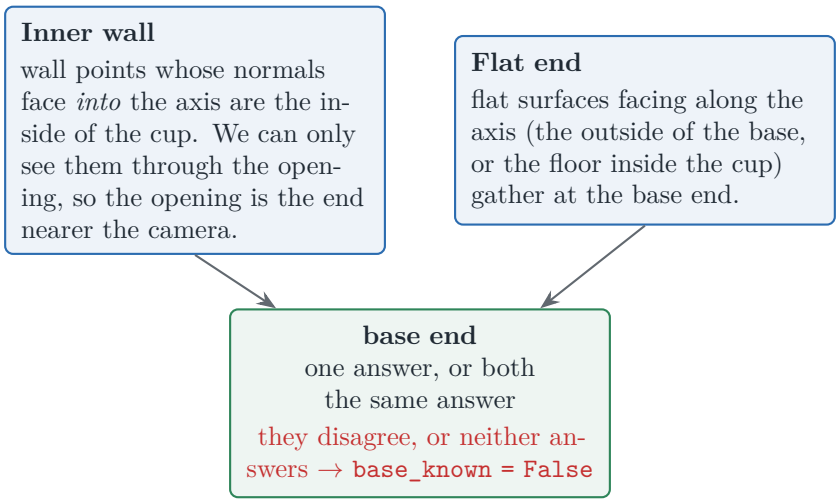
## 11 Step 6: which end is the base, which way is the handle

A mug standing on the table gets its base from the table (section 8). A mug lying down, or held in the air, does not touch the table where we can use it. Without knowing which end is the base, its axis could point either way, and a grasp or a re-stand would turn it upside down.



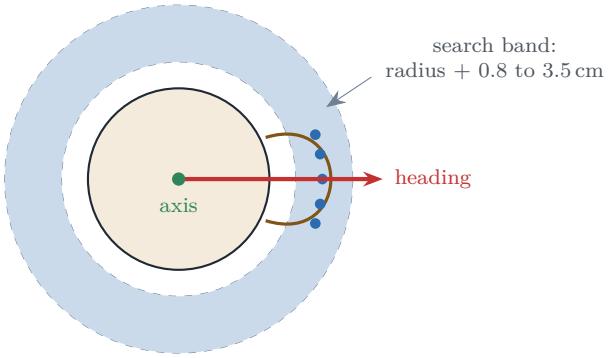
## 11.1 Two cues for the base end, which must agree




The inner-wall cue held in all 221 views on seeds 0–4 where at least a tenth of the wall faced inwards. The flat-end cue alone failed on lying mugs, which is why there are two.

## 11.2 Handle heading

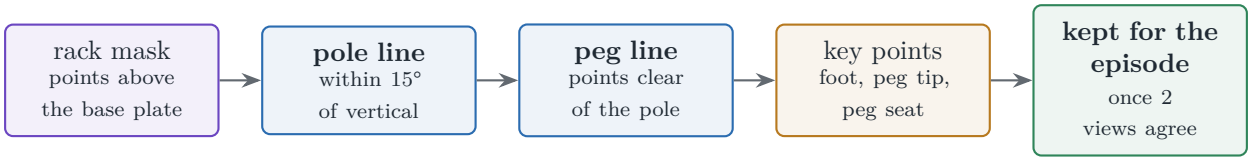
Handle points lie a little *outside* the body: 0.8–3.5 cm beyond the fitted radius, within the body's height, all on one side. Their mean direction from the axis is the `heading`. If they point in different directions (agreement below 0.8), no heading is given. This test has a known weakness when the handle is hidden (section 15).

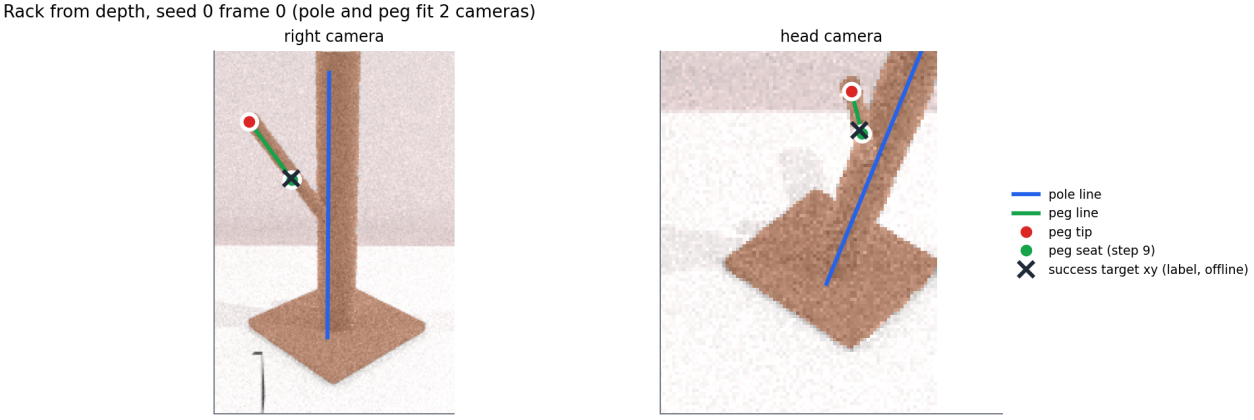



| | Seeds 0–4 | Seeds 5–9 |
|---|---|---|
| Lying mugs with a known base end | 25 of 28 | 28 of 28 |
| Base at the wrong end | 0 | 0 |
| Frames with a heading; median error (worst) | 66; 2.8° (11.9°) | 117; 2.8° (5.5°) |

## 12 Step 7: the rack from depth

The rack is a vertical pole on a base plate with one slanted peg. That is two straight lines, so the fit is two robust (RANSAC) line fits on the rack mask's points above the plate:




- **Peg tip**: the far end of the peg line. Median error 2.9 mm (seeds 0–4) and 4.2 mm (seeds 5–9), worst 1.4 cm.
- **The rack does not move**, so its pose is fitted until it succeeds and then kept. After the review (section 13) it is kept only once **two views agree** on the peg tip within 2 cm: two cameras in the same frame, or the same camera in two frames. One bad view (an arm across the peg, a stray mask) cannot fix a wrong hang target for the whole episode. It arrives at frame 0 or 16 in 9 of 10 episodes.

## 13 A second look: the code review

Before wiring poses into the robot's targets, the branch was reviewed as a whole. What changed:

| Finding | Change |
|---|---|
| One rack fit decided the episode | two views must agree before the pose is kept |
| No table height, still a rack pose | no table, no rack pose: the plate's edge could pass for the peg |
| Key points could be edited | read-only, because one cached rack pose is shared by every later frame |
| Empty table mask crashed | guarded; the pose says *unknown* |
| Normals computed on the whole image | only on the mask's bounding box: identical output on 320 real masks, 1.8× faster |

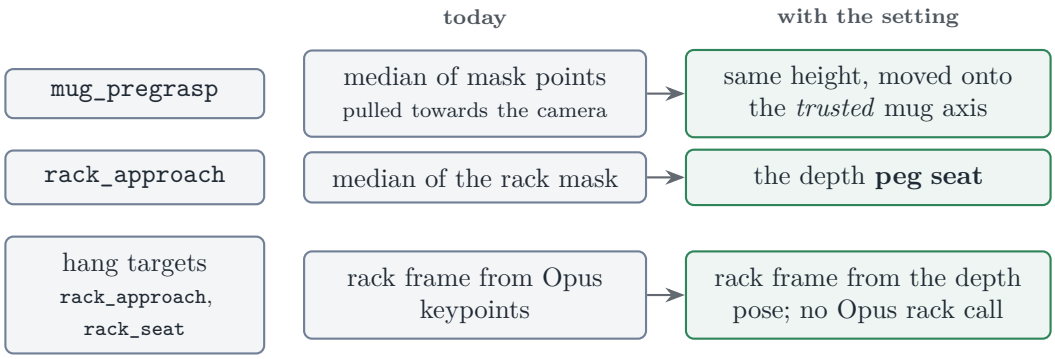
Cost per frame after the review: about **32 ms** for the mug pose; the rack fit, about **164 ms**, runs only until the rack pose is kept.

# 14 Step 9: aiming the arm with the poses

Steps 1–7 only *describe* the scene. Step 9 lets the poses decide *where the arm goes*. Saved frames cannot show whether a different target makes a grasp or a hang succeed, so this is behind a setting that is **off by default**. Pose estimation is judged by its accuracy (section 15); whether the new targets raise hanging success needs a live comparison, which is follow-up work on the supervisor side. The rack results here were all measured with the `explore_mug_rack` prompts, so that profile belongs with the setting:

```
perception:
rgbd_enabled:  true
depth_pose_anchors:  true   # needs --perception-factory
rack_prompt_profile:  explore_mug_rack
```

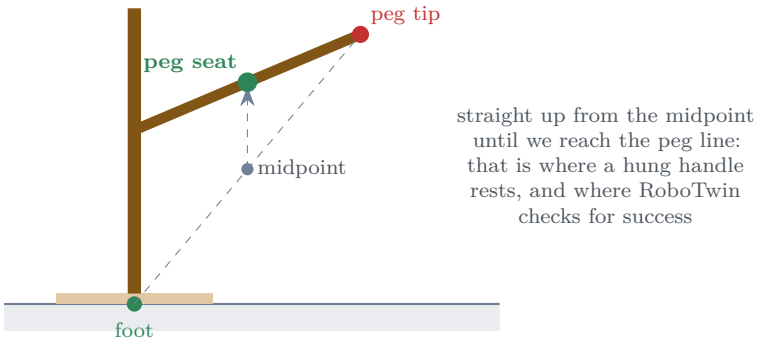
## 14.1 What moves

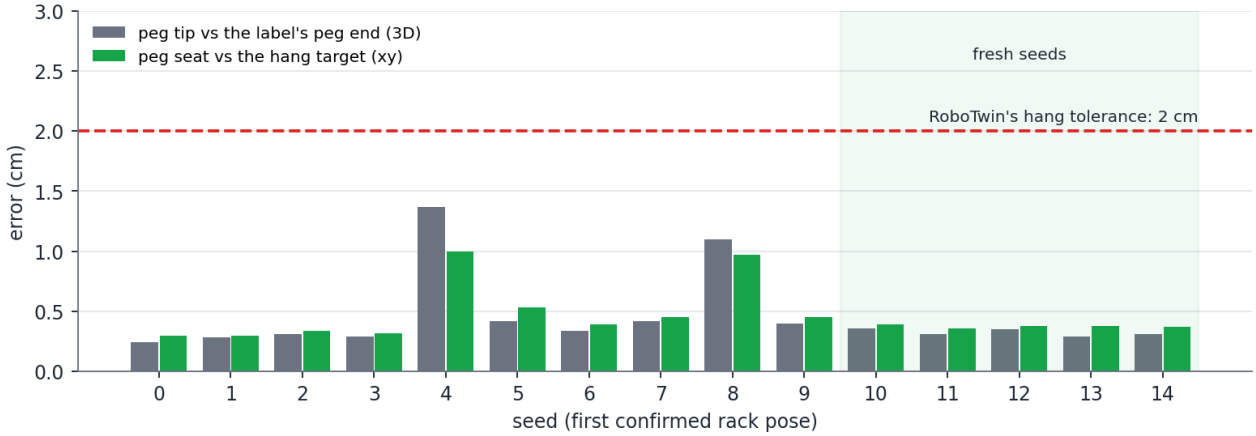



Every new target **falls back** to today's target when its pose is missing (no trusted mug pose, no confirmed rack pose). The mug-handle frame for the hang still comes from Opus.

## 14.2 The peg seat: aim at what success is measured by

RoboTwin counts a hang as done when the mug's handle is within 2 cm (in xy) of the **midpoint of the rack's base and the peg's end**, above a height floor, with the gripper open. The peg *tip* is the wrong place to aim: it is 2.4–3.5 cm from that target. The seat is the point *on the peg* above the middle of the pole's foot and the peg tip.




## 14.3 The rack frame for the hang

The hang code already turns two frames (rack peg, mug handle) into gripper targets. The depth rack frame copies the Opus keypoint frame's convention exactly, so that code did not change: origin at the seat, $+x$ along the peg to its tip, $+y$ up the pole.

**Careful**

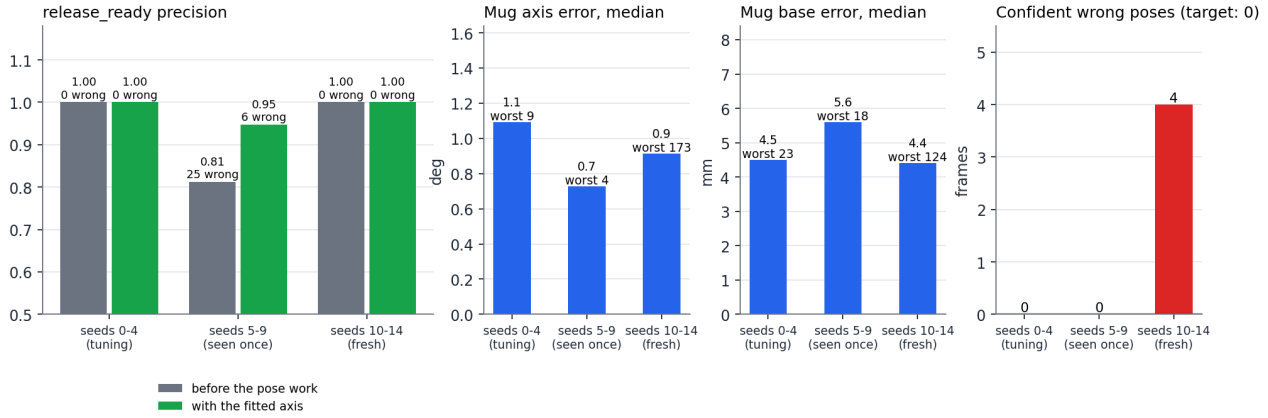
**Why the handle frame stays with Opus.** The depth handle point (the median of the handle's visible points) is 1.7–2.1 cm from the label's handle point, and a hang must land within 2 cm. It is tracked in the evaluator (`handle_point_error_m`) for later work, not used.

**Where it lives in the code**

- `object_pose.py`: `_peg_seat`, the `peg_seat` key point.
- `grounding.py`: `bind_pose_approach_anchor`; `episode_rack_pose`.
- `hang_targets.py`: `rack_frame_from_pose`, `DepthRackFrame`.
- `pose_grounding.py`: `frame_from_axes`, shared with the Opus keypoint path.
- `preparation.py`, `runtime/settings.py`, `runtime/assembly.py`, `run_skills.py`: the setting.

# 15 Step 8: a clean test on fresh seeds

Seeds 0–4 were used for tuning and seed 6 was inspected while designing the trust rule, so neither can say whether the method holds up. Seeds 10–14 were captured after the code was frozen at commit `4257459`, and scored once. **Nothing was changed after seeing them.**



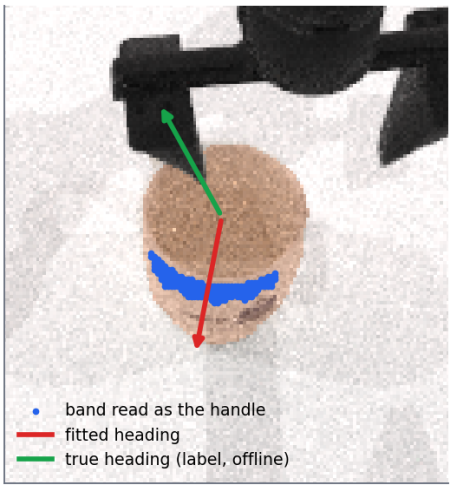

## 15.1 Against the targets set in the plan

| Measure (seeds 10–14, 290 frames) | Target | Result | |
|---|---|---|---|
| Mug base error, median / 90th percentile | 1.5 / 3 cm | 0.44 / 1.66 cm | ✓ |
| Mug axis error, median | 10° | 0.9° | ✓ |
| Upright/toppled mix-ups | 0 | 0 | ✓ |
| Handle heading error, median | 20° | 2.9° | ✓ |
| Rack peg tip error, median (worst) | 1 cm | 3.1 mm (3.6 mm) | ✓ |
| **Confident wrong poses** | **0** | **4** | ✗ |

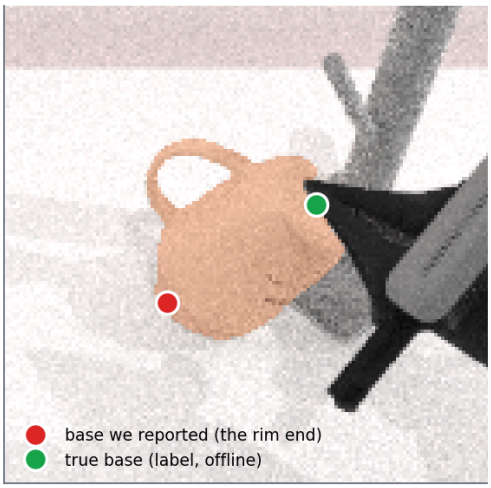
The plan said the confirmation must state how much toppled evidence it holds: **71 toppled frames** (seed 10: 19, seed 13: 29, seed 14: 23), with the base end known in 48 of them. The median base error on those is 1.6 cm, larger than the 4–6 mm over all poses, because a lying mug's base is placed where its visible points end.

## 15.2 The four failures

Rim read as a handle (seed 11, frame 208)
the real handle is out of view, behind the mug




Upside down while tipping (seed 13, frame 464)
true tilt 146 deg; the axis is within 45 deg of vertical, so it was read as 'up'



| Frames | What happened | Effect today |
|---|---|---|
| 3 frames: seeds 11 and 13, mug model 4 | With the handle out of view, a band of points near the rim sits just outside the fitted radius, on the camera side. It reads as a handle pointing at the camera: 145–168° off. Axis and base are correct (3 mm). | None: no decision reads the heading yet. A recovery skill choosing a grip side from it would be misled. |
| 1 frame: seed 13, frame 464 | The mug was upside down while tipping (146°). Its axis was within 45° of vertical, and the code assumes such a mug has its base at the bottom. The base landed on the rim end, 12 cm off. | None: the frame was not called safe to release, and the mug approach uses the axis *line*, which is unaffected by the flip. |

**Careful**

**These are not patched.** A fix made after looking at fresh seeds would spoil them as a test, which is what happened to seed 6. The proposed fixes are below (section 17); they should be tuned on seeds 0–4 and confirmed on a new set of seeds.

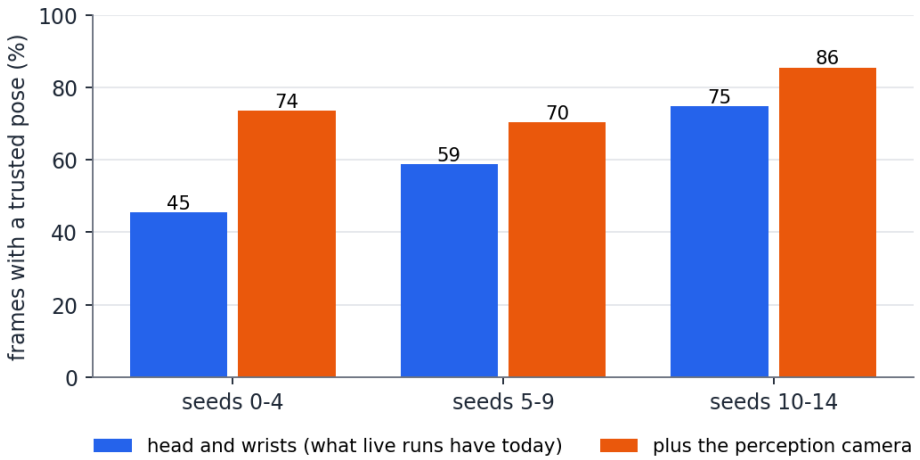
## 15.3 The release check on fresh seeds

| Seeds 10–14 | With perception camera | | Head and wrists only | |
|---|---|---|---|---|
| | before | after | before | after |
| `release_ready` precision | 1.00 | 1.00 | 1.00 | 1.00 |
| `release_ready` recall | 0.61 | 0.62 | 0.52 | 0.52 |
| `mug_staged_on_table` recall | 0.53 | 0.54 | 0.47 | 0.47 |
| `mug_lifted` recall | 0.86 | 0.86 | 0.74 | 0.74 |
| `right_mug_lifted` recall | 0.95 | 0.95 | 0.53 | 0.53 |
| False “true”, any predicate | 0 | 0 | 0 | 0 |

The pose work did no harm, but this test **neither confirms nor refutes** the earlier gain (25 → 6 wrong on seeds 5–9): none of the fresh seeds produced the end-on mug that fools the old surface test, so there was nothing for the veto to catch. That gain still rests on seeds we looked at.

## 15.4 Which cameras a live run actually has

The “with perception camera” numbers need a caveat. The fixed perception camera exists only in the capture tools; `run_skills.py` does not add it. Each observation, SAM 3 looks at the **head camera**, and tries another camera only if the head finds no confident mug (the wrist for the phase) or, with the `explore_mug_rack` profile, no clear rack (the right wrist). **Today’s live runs match the blue bars.**

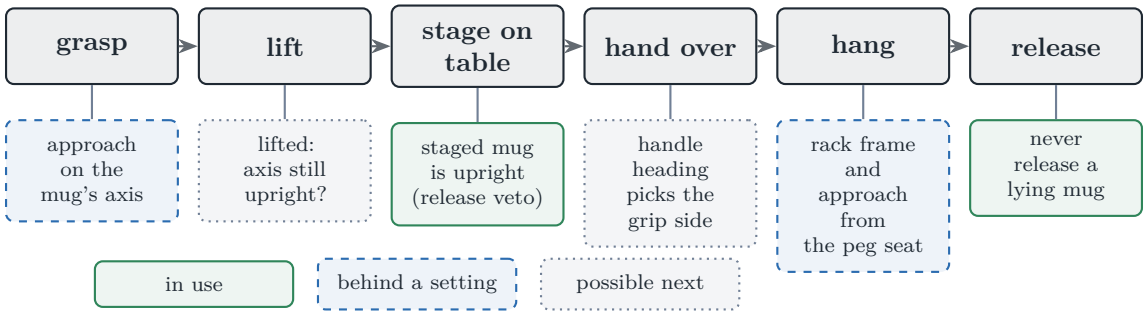


Adding the camera mainly buys coverage. Replaying each order on the cached answers (483 frames, seeds 0–9, mug detections only) shows it hardly changes the number of SAM 3 calls:

| Camera order | Calls per frame | Mug found |
|---|---|---|
| Head, then wrist (today) | 1.69 | 61% |
| Head, then perception, then wrist | 1.75 | 82% |
| Perception, then head, then wrist | 1.62 | 82% |
| Perception, then wrist | 1.51 | 74% |

## 16 What this means for supervising the task

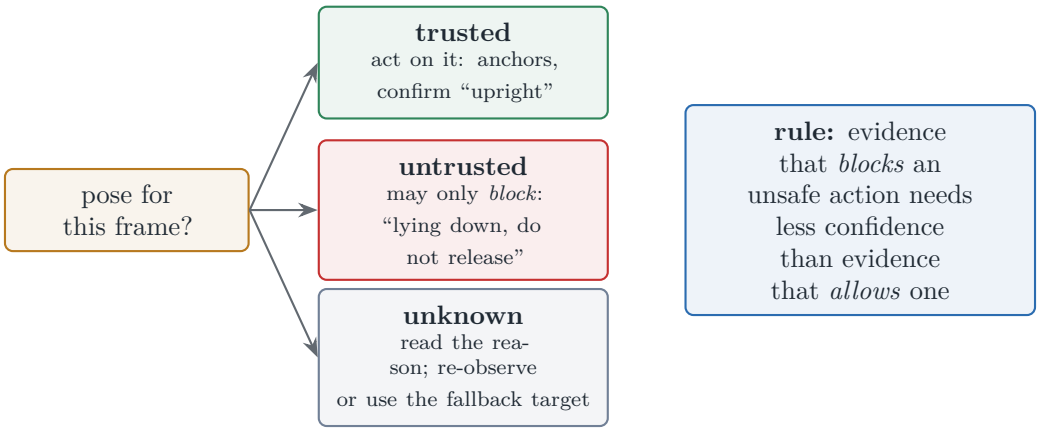
The supervisor does not move the robot itself. It watches, decides and checks: the planner picks the next skill (move to an anchor, grasp, release), the frozen policy runs it, and predicates read the cameras to say whether a phase is done or whether it is safe to go on. Supervision is only as good as what it can *see*. Before this work it saw "a mug-coloured blob, roughly here". Now it sees an object with a base, an axis, a handle and a rack peg, and it knows when it does not know.




### 16.1 Five things the supervisor gains

1. **A release gate that is harder to fool** (measured). A wrong "safe to release" is the worst predicate mistake: the mug drops or tips, and the task usually cannot recover. The fitted axis vetoes it even when the fit is only partly trusted (section 7). Wrong answers fell from 25 to 6 on seeds 5–9; on the fresh seeds there were none before or after (section 15).
2. **Failure states it can recognise** (measured). "The mug has toppled" is now read from the axis: no upright/toppled mix-ups on any of the 15 seeds, and on the fresh seeds the base end is known in 48 of 71 toppled frames. A recovery skill could use this, but none does yet, and a lying mug's handle is rarely seen.
3. **Targets that match what is checked** (accuracy measured, benefit expected). RoboTwin judges the hang at the peg seat, not the peg tip. The seat is 3–5 mm from the success target, where the tip is 2.4–3.5 cm from it. The mug approach no longer aims at a point pulled 2 cm towards the camera. Whether this raises hanging success has not been run.
4. **One model call fewer** (behind the setting). The rack frame for the hang came from Opus keypoints: a network call to a large model, with its latency and cost. With the setting on, it comes from depth, about 0.16 s per try until two views agree, and is then kept for the episode.
5. ***Unknown* with a reason.** Every pose carries `trusted` and a `reason` (`cameras_disagree`, `cylinder_fit_inliers_low`, `rack_fit_awaiting_confirmation`, ...). The supervisor can respond to the reason: look again from another camera, wait for the arm to clear the view, or fall back to the old target, instead of acting on a guess.

### 16.2 How the supervisor should read a pose



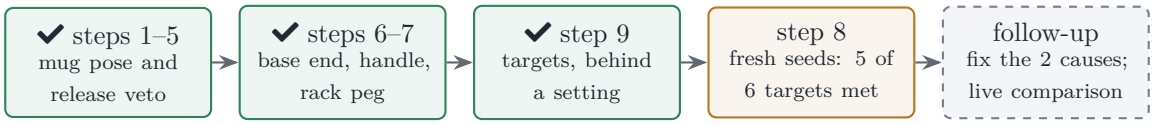

### 16.3 What the planner can and cannot infer

| ✔ Can | ✖ Cannot (yet) |
|---|---|
| whether the mug stands or lies, and where its base is (mm) | the mug's pose while a gripper hides most of it: only 40–60% of right-hand-held frames get a trusted pose |
| which end of a lying mug is the base | the handle's exact position: about 2 cm off, too coarse to hang by |
| the handle direction of a standing mug when the handle is in view (about 3°) | whether the handle is *on* the peg: the hang itself is still checked by RoboTwin; nor a hidden handle's direction, which can be misread |
| where the hung handle must end up on the rack (3–5 mm) | which end is the base of an upside-down mug near vertical; anything about objects with no spec |

## 17 What is still open

**Careful**

- **The "no confident wrong poses" target failed on fresh seeds** (4 of 290 frames). Neither cause affects a decision today. Both need fixing before anything relies on them:
  - *Rim read as a handle:* require the handle points to span part of the mug's height, which a thin rim band does not.
  - *Upside-down near vertical:* use the base-end cues on tilted mugs too, and report *base unknown* when they disagree with "up".
- **The release gain is unconfirmed.** The 25 → 6 result is from seeds 5–9, and the fresh seeds held no case that tests it. Its 6 remaining wrong answers are all seed 6's end-on mug.
- **Held mugs are often unknown.** Gripper points are not removed, because the measured hand pose sits about 0.2 m behind the fingers and a box around it would delete the mug. On the fresh seeds, 59% of right-hand-held frames get a trusted pose.
- **The depth handle point is too coarse to hang by**: 2.9 cm median on the fresh seeds. The handle frame stays with Opus.
- **Live results do not exist.** `depth_pose_anchors` has never run in a live episode. The rack may also be confirmed late (seed 10: frame 112), and targets fall back until then.




## 18 Commands

Run from the repository root. Scoring needs no GPU once the SAM 3 answers are cached.

**Tests** (CPU, seconds):

```
conda run -n robotwin python -m pytest -q tests/test_object_pose.py \
tests/test_perception_grounding.py tests/test_perception_scoring.py
```

**Score poses and predicates** (under a minute per five seeds):

```
conda run -n robotwin python -m development.replays.perception_evaluation \
--frames data/perception/frames_cam/seed_0 \
...  (one --frames per seed, up to seed_4) \
--output data/perception/reports/pose_dev_with_camera
```

Add `--ignore-camera perception_camera` to score without the new camera. The report's `summary.md` has a "Mug pose by state" table; `frames.jsonl` has the per-frame pose errors and reasons.

**Repeat the fresh-seed test.** Cache each capture once (it resumes if interrupted), then score from a checkout of the frozen commit `4257459`:

```
python -m development.replays.sam3_cache --frames data/perception/frames_fresh/seed_10
python -m development.replays.perception_evaluation \
--frames data/perception/frames_fresh/seed_10 ...  (up to seed_14) \
--output data/perception/reports/fresh_with_camera
```

**Rebuild this guide**:

```
conda run -n robotwin env PYTHONPATH=$PWD python \
docs/Collaboration/Olusola/guides/pose_estimation/make_figures.py
cd docs/Collaboration/Olusola/guides/pose_estimation && \
conda run -n texbuild tectonic -X compile pose_estimation_guide.tex
```

**Where it lives in the code**

Read next: `plans/pose_estimation_plan.md` (the full plan and its progress notes) and `guides/perception/perception_guide.pdf` (how predicates and the provider work).